Software Requirements Specification

Field	This example
Team name	**Viounique**
Project title	**Assistance application for Disasters**
SDLC chosen in M1 + reason	**We selected the Agile SDLC for its flexibility to handle evolving requirements and support continuous future development.**
Date / version	**September 4, 2026**
GitHub repo	https://github.com/Viounique

1. Introduction

1.1 Purpose and scope

In 2025, Myanmar was struck by catastrophic flooding driven by intense monsoon rains and regional cyclones. Low-lying river basins and coastal regions were submerged, leaving tens of thousands of residents trapped, critical infrastructure damaged, and communication networks severed. Rescue agencies struggled to locate victims in urgent need and distribute life-saving supplies efficiently.

In 2025–2026, Thailand faced severe, recurring floods across both northern river basins and Southern urban hubs. Densely populated communities and remote mountain villages were suddenly cut off by flash floods and mudslides. Emergency responders were overwhelmed by fragmented, delayed reports, making it nearly impossible to prioritize vulnerable populations such as the bedridden, elderly, and infants.

These regional disasters reveal a critical, shared vulnerability: the lack of a unified, intelligent system for emergency reporting and crisis coordination. Victims are unable to send real-time location data or specify special care needs, coordinators lack live operational visibility, and rescue teams are forced to deploy blindly. Precious hours are lost, and lives are endangered simply because critical information fails to flow when it is needed most.

1.2 Users and stakeholders

Name/role	Kind	What they need	What they fear/forbid
Flood Victim / Affected Citizen	primary user	Simple way to send GPS coordinates and request special medical/food aid without complex setup	Losing signal and requests failing silently; delay in critical rescue teams reaching them
Field Rescue Team/ First Responder	primary user	Clear priority triage (Color Codes) and exact locations of vulnerable groups (e.g., bedridden, infants)	Wasting resources on fake/duplicate reports; entering dangerous flood zones without proper equipment

Name/role	Kind	What they need	What they fear/forbid
Emergency Rescue Coordinator/ Operations Center	Secondary stakeholder	Real-time live map visibility, centralized data from local/national agencies, and efficient resource allocation	System overload or crashes during peak disaster hours; fragmented, conflicting information leading to missed cases
Government Agencies & NGOs (e.g., ปภ., WFP, UNICEF)	Institution/ Partner	Standardized data integration, transparent rescue logs, and secure access to demographic/needs data	Data privacy breaches; personal health or identity data leaking into unverified public channels

2. Overall description

2.1 Product context — what it does/does not

IN SCOPE (MVP this term)	OUT OF SCOPE (explicit promise)
• FR-1 Victim Request Logging: Log distress requests via the app from mobile. • FR-2 Capture requests offline and sync when online. • FR-3 Rescue Coordinator Live Map: Display requests on a real-time map with filters. • FR-4 Priority Scoring (Rule-based): Rule-based scoring to quickly prioritize requests. • FR-5 Priority Scoring (Rule-based): Rule-based scoring to quickly prioritize requests (AI).	• No group chat or direct text messaging inside the app. • No payment processing or donation subscription feature. •No AI tutor or medical assistant to answer clinical questions. • No reading or summarizing external LINE/Messenger groups.

2.2 Assumptions and constraints

- Assumption: Every victim or volunteer has a smartphone capable of capturing GPS coordinates and connecting to cellular or internet networks.
- Constraint: One LLM API call per natural-language request parsing — no custom model training, no fine-tuning.
- Constraint: Thai and English UI text; no paid API beyond the single LLM call above.
- Constraint: Personal data — we store victim name, phone number, GPS location, and emergency type only; we do not store message content from any chat app.
- Constraint: 10-week build window shared with four other milestones (M1–M5).

3. Functional requirements

Format: As a <role>, I want <goal>, so that <benefit>. Every story below traces to a pain observed in a (simulated) student interview, and every acceptance criterion is something a test could actually check.

FR-1 — Victim Request Logging

User story: As a flood victim, I want to log a distress request with my GPS location and assistance type, so that rescue teams can locate and reach me with appropriate help.

Pain this traces to: Victims struggle to describe their location over phone calls or lose signal before completing complex forms, causing rescue teams to miss their exact whereabouts.

1. Given GPS location access and emergency type entered, when the victim submits the request, then the request appears on the system with those coordinates and details.
2. Given an existing distress request, when the victim edits their details or contact phone number, then the updated information is shown immediately, and no duplicate entry is created.

FR-2 — Data Synchronization (Offline/Online)

User story: As a flood victim, I want my distress request captured offline and automatically synced when the connection is restored, so my request is never lost due to an unstable network.

Pain this traces to: Unstable mobile networks in disaster zones cause online forms to fail silently, leaving victims unaware that their call for help was lost.

1. Given no internet connection, when the victim submits a request, then the request is saved in local storage with an "offline pending" status.
2. Given a pending offline request, when the device reconnects to the network, then the request automatically syncs to the server and changes status to "submitted".

FR-3 — Rescue Coordinator Live Map

User story: As a rescue coordinator, I want to see all active requests on a real-time interactive map with filters, so that I can visualize priority areas and allocate resources efficiently.

Pain this traces to: Coordinators receive fragmented reports from various channels without spatial context, making it hard to identify heavily affected disaster clusters.

1. Given active emergency requests on the server, when the coordinator opens the map view, then all requests are plotted as pin markers at their recorded GPS coordinates.
2. Given requests displayed on the map, when the coordinator applies a filter (e.g., "Medical Emergency Only"), then only matching markers remain visible on the map.

FR-4 — Priority Scoring (Rule-based)

User story: As a rescue coordinator, I want the system to calculate an initial priority score using predefined safety rules, so that urgent medical or vulnerable cases are immediately highlighted.

Pain this traces to: High volumes of incoming distress calls overwhelm rescue centers, causing life-threatening cases to be buried under non-urgent requests.

1. Given a request tagged with a vulnerable condition (e.g., bedridden patient or infant), when processed by the system, its priority score is set to high priority (Red code).
2. Given a request marked for non-urgent supplies (e.g., dry food only), when processed, then its priority score is set to lower priority (Green or Yellow code).

FR-5 — Communication & Contact

User story: As a rescue coordinator, I want to initiate direct contact with the victim via their recorded phone number. So that I can verify their condition before dispatching a rescue team.

Pain this traces to: Rescue teams waste critical time navigating flooded zones only to find that the victim has already evacuated or provided incorrect details.

1. Given a distress request on the coordinator dashboard, when the coordinator clicks the contact button, then the system triggers a call link using the victim's stored phone number.
2. Given an unconfirmable or false request, when the coordinator updates the case status to "Unreachable / Canceled", then the updated status is reflected across the dashboard

4. Non-functional requirements

Forbidden words unless quantified: fast, easy, user-friendly, smart, secure. NFR-5 (security/privacy) is required for M2 — every SRS must have at least one.

ID	Kind	Statement (testable)	How we will check
NFR-1	performance	The real-time map view must load and display all emergency incident markers within 3 seconds on a standard 4G mobile network.	Stopwatch / Network profiler on a test smartphone, average of 3 trials.

ID	Kind	Statement (testable)	How we will check
NFR-2	performance	A distress status update notification shall arrive on the victim's phone within 30 seconds of being updated by a coordinator.	Compare the server update timestamp against the notification receipt timestamp on the receiving device.
NFR-3	usability	A first-time victim shall log a complete distress request (GPS + emergency type) in under 45 seconds without external guidance.	Timed trial with a user who has not seen or used the app before.
NFR-4	reliability	The offline request logger shall retain all unsynced data locally without data loss if the app crashes or the phone restarts.	Force-close the app and restart the device during offline pending state, then verify that data syncs upon reopening.
NFR-5	security/privacy (required)	A coordinator shall access only distress requests within their authorized sector; requesting another sector's request ID directly via API shall fail.	Modify the request ID or sector ID parameter in an API call and confirm it returns a 403 Forbidden error.

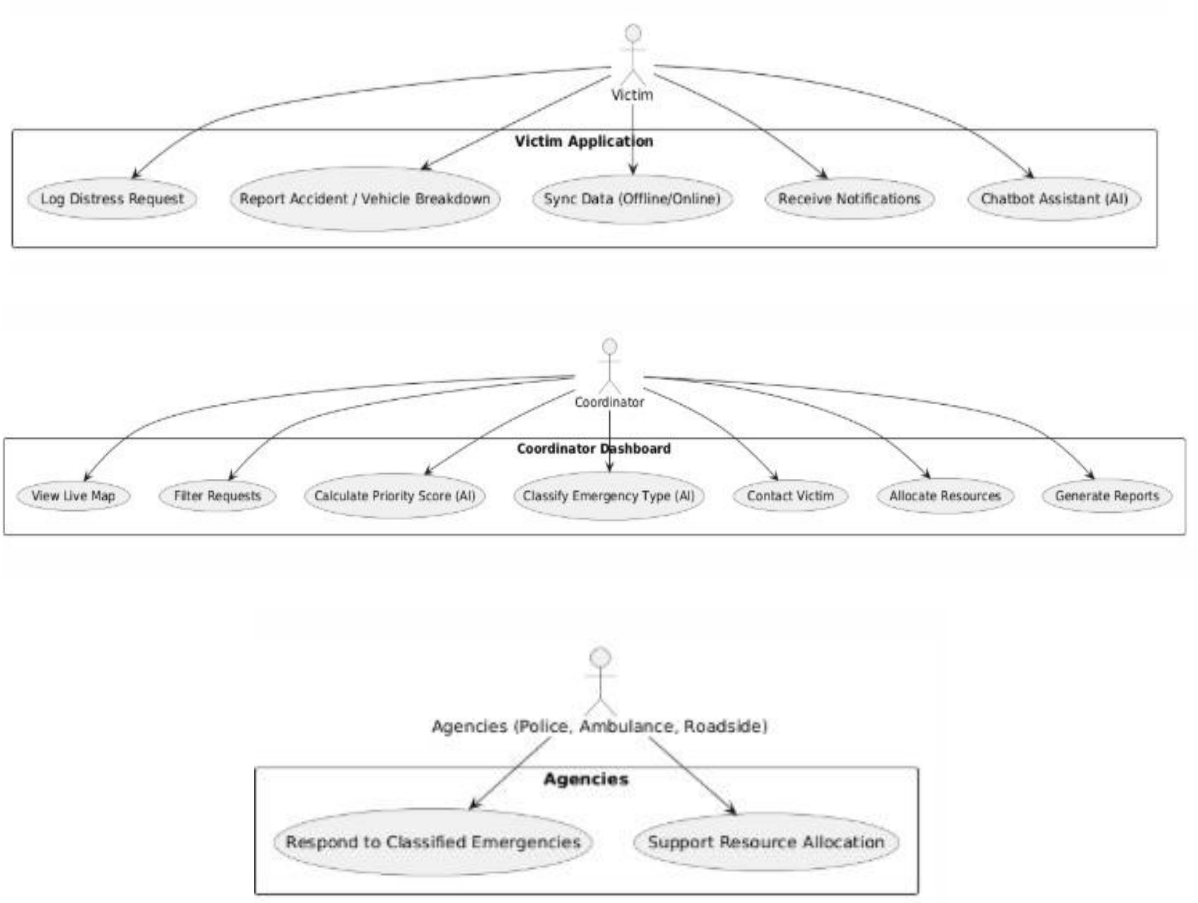
5. Use cases

5.1 Use-case list

Use case	Actor	Related FR	Brief success story
UC-1 Log Distress Request	Victim	FR-1	Victim logs emergency request with GPS coordinates; it appears in the system queue.
UC-2 Report Accident / Vehicle Breakdown	Victim	FR-8	Victim submits a general emergency report; the system categorizes it separately from disaster cases.
UC-3 Sync Data (Offline/Online)	Victim Application → Server	FR-2	App detects restored internet connection and automatically syncs pending offline requests.
UC-4 Receive Notifications	Victim	FR-12	Victim receives real-time status alert on their phone when rescue team is dispatched.
UC-5 View Live Map	Coordinator	FR-3	Coordinator opens map dashboard and views all active distress markers clustered by region.
UC-6 Filter Requests	Coordinator	FR-3	Coordinator applies filters (e.g., medical cases) to isolate specific emergency categories.
UC-7 Calculate Priority Score (AI)	Coordinator / AI Module	FR-4, FR-13	System/AI evaluates incoming report details and assigns initial priority score (e.g., red status).
UC-8 Classify Emergency Type (AI)	Coordinator / AI Module	FR-9, FR-14	AI parses natural language text from the report and automatically selects incident type.
UC-9 Contact Victim	Coordinator	FR-5	Coordinator initiates a direct call via dashboard to verify victim status before dispatching help.
UC-10 Allocate Resources	Coordinator	FR-6	Coordinator assigns rescue teams and available equipment to high-priority cases.
UC-11 Generate Reports	Coordinator	FR-7	Coordinator exports analytical summaries of disaster relief operations for agency review.

Use case	Actor	Related FR	Brief success story
UC-12 Respond to Classified Emergencies	Agencies	FR-8, FR-9	External agencies (Police/Ambulance) receive classified tasks and deploy field teams.
UC-13 Support Resource Allocation	Agencies	FR-6	Partner agencies provide additional rescue equipment/personnel to support operations.

5.2 Use-case diagram



6. Out of scope

1. No internal text chat capabilities between victims, volunteers, or rescue personnel.
2. No AI-driven medical diagnostic tools or clinical advice capabilities to answer health and treatment questions.
3. No integration to read, fetch, or summarize messages from external chat platforms or social media groups.

7. Traceability matrix (Golden Thread)

Only FR-3 is filled all the way across — that is the single thread this course follows week by week (Ch3 → Ch4/5 → Ch6 → Ch7). The other rows stay open until their week arrives, exactly like your own M2 should.

Problem/pain	Requirement (FR)	Solution/design	Feature built	Test
Victims struggle to describe locations; teams miss exact coordinates.	FR-1 Victim Request Logging	(Week 5)	(Week 6–7)	(Week 7)
Students lose track of Unstable networks, causing silent failures; requests are lost and still pending	FR-2 Data Synchronization (Offline/Online)	(Week 5)	(Week 6–7)	(Week 7)
Coordinators lack spatial context to identify high-risk clusters.	FR-3 Rescue Coordinator Live Map	Request entity + Real-time map rendering job (Ch4–5)	"Live Rescue Map" dashboard view (Ch6)	Fetch_active_requests_map() unit tests (Ch7)
One deadline at a time. Incoming request volume overwhelms centers; critical cases are buried. Missed same-day clashes	FR-4 Priority Scoring (Rule-based)	(Week 5)	(Week 6–7)	(Week 7)
Teams waste time navigating flooded zones to false or changed locations.	FR-5 Communication & Contact	(Week 5)	(Week 6–7)	(Week 7)

8. AI usage log

If this table is empty but the prose looks generated, the milestone is capped at 50% — log every prompt that drafted a story or paragraph, honestly.

Date	Tool / model	What we asked	What AI produced	What we changed/ verified	What we learned
5 September 2026	Gemini	"Analyze recent flooding disasters in Southeast Asia (Myanmar 2025, Thailand 2025–2026) and outline key operational bottlenecks faced by rescue agencies and victims."	A detailed overview of regional flooding impacts, highlighting issues like severed communications, uncoordinated rescue efforts, and delays in identifying high-risk victims (elderly, infants).	Synthesized the generic summary into a structured 1.1 Purpose and scope section; filtered out macro-economic statistics to focus strictly on actionable software-level pains (e.g., lack of real-time GPS coordinates and live operational visibility).	AI provided strong historical background context, but contained overly general commentary; we learned to distill raw AI background research directly into concrete "pain points" that trace directly to functional requirements. .

9. Self-check

- ☑ Every FR is a user story with a benefit — all 5 FRs in §3 end in a “so that” clause tied to a real pain.
- ☑ Every FR has acceptance criteria you could actually test — 2 AC per FR, each a number, a state, or a yes/no.
- ☑ No FR is vague — no bare “fast”, “easy”, or “user-friendly” anywhere in §3.
- ☑ NFRs exist, including security/privacy — NFR-5 is a required access-control check.
- ☑ Every FR traces back to a stated problem — see “Pain this traces to” under each FR, and §7.
- ☑ Scope and out-of-scope are stated — §2.1 and §6.
- ☑ Use-case list matches the FR list — 5 use cases, one per FR (diagram added Week 4).

☑ AI log is honest — §8 shows exactly what was asked, drafted, and hand-corrected.

Course rule: you may use AI to draft. You must log it, verify it against a real stakeholder, and be able to explain every FR in the oral exam. Work you cannot explain earns no credit.